Today 11:24 AM

hey im working on a pygame tic tac toe lab from a github project(https://github.com/SETAPESU26/09_tic_tac_toe) where i play X and the computer plays O. there are 3 problems with the win draw logic.
diagonal wins dont get detected at all. if my winning move also fills the last empty cell it says draw instead of showing the winner. and after a round ends u can still click empty cells and new symbols keep appearing.
can u fix all 3 things and just give comeplete updated files back

Absolutely. I checked the **actual starter code from your GitHub repo**, so these changes match its existing structure rather than giving you a generic Tic-Tac-Toe implementation. The three bugs are exactly where you described: diagonals are missing from `rules.py`, `check_round_end()` checks for a full board before checking for a winner, and `handle_click()` doesn't stop input after the round ends. GitHub +1

You only need to replace **these 2 files**. `main.py`, `ai.py`, and `renderer.py` can stay exactly as they are. GitHub +2

`tic-tac-toe/game/rules.py`

Python

```
"""
rules: win/draw detection for a 3x3 board.

board is a 3x3 list of lists, each cell is None, 'X', or 'O'.
"""


def check_winner(board):
    lines = []

    # Rows
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])

    # Columns
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])

    # Main diagonal: top-left -> bottom-right
    lines.append([board[0][0], board[1][1], board[2][2]])

    # Other diagonal: top-right -> bottom-left
    lines.append([board[0][2], board[1][1], board[2][0]])

    # Check all 8 winning lines
    for line in lines:
        if line[0] is not None and line[0] == line[1] == line[2]:
            return line[0]

    return None
```

`tic-tac-toe/game/game_engine.py`

Python

```
"""
GameEngine: owns the board, turn state, and round-end logic.

You (the player) always play X and click to move.
The computer always plays O and moves automatically right after you,
using a simple random-move AI.
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move


HUMAN_SYMBOL = 'X'
```

```
COMPUTER_SYMBOL = 'O'


class GameEngine:
    def __init__(self):
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = HUMAN_SYMBOL
        self.round_over = False
        self.winner = None   # 'X', 'O', or None for a draw

    def handle_click(self, pos):
        # Do not allow any moves after the round has ended
        if self.round_over:
            return

        # Only allow the human to move when it is X's turn
```

What was fixed

1. Diagonal wins

Both missing diagonals are now included:

```
X . .
. X .
. . X
```

and

```
. . X
. X .
X . .
```

So all **8 winning combinations** are checked. GitHub

2. Win vs draw

The old code did this:

```python
if is_board_full():
    return draw

check_winner()
```

So if the final move created a winning line, the board was detected as full first and incorrectly called a draw. The new code does:

```python
winner = check_winner()

if winner:
    # WIN

if is_board_full():
    # DRAW
```

Therefore **a win always takes priority over a draw**. GitHub

3. No clicking after game ends

This is now the first check in `handle_click()`:

```python
```

```
if self.round_over:
    return
```

So once `"X wins!"`, `"O wins!"`, or `"Draw!"` appears, clicking empty cells does absolutely nothing.

I also added the occupied-cell check:

```python
if self.board[row][col] is not None:
    return
```

which fixes the additional move-validation bug already identified in the lab. GitHub

One important detail: after X wins, the code now immediately returns, so it **doesn't switch to O or let the computer make another move**. Same for O winning. This keeps the round properly frozen.

bugs are fixed now i want to add a persistent scoreboard on screen showing X wins O wins and draws. it should stay the same when i start a new round and only clear when i reset the whole match. lets use R for a new round and M to reset the match.
can u add this and just give comeplete updated files back

Yep. Since your current win/draw logic is already fixed, we only need to add **match-level state** separate from the current round.

Replace your current `game_engine.py` with this. Your `rules.py` does **not** need any changes.

`tic-tac-toe/game/game_engine.py`

```python
"""
GameEngine: owns the board, turn state, round-end logic,
and persistent match scoreboard.

Player = X
Computer = O

R = New round
M = Reset entire match
"""

import pygame

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move


HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'


class GameEngine:
    def __init__(self):
        # Current round
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = HUMAN_SYMBOL
        self.round_over = False
        self.winner = None
```

`tic-tac-toe/game/rules.py`

No scoreboard changes are needed here, so keep your current fixed version:

```python
"""
rules: win/draw detection for a 3x3 board.
```

```
board is a 3x3 list of lists, each cell is None, 'X', or 'O'.
"""


def check_winner(board):
    lines = []

    # Rows
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])

    # Columns
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])

    # Main diagonal
    lines.append([
        board[0][0],
        board[1][1],
        board[2][2]
    ])

    # Other diagonal
    lines.append([
        board[0][2],
        board[1][1],
```

One thing to check in `main.py`

Your event loop needs to pass **keyboard events** to `handle_keydown()`. If your existing `main.py` already has this, you don't need to change it:

‹/› Python

```
if event.type == pygame.KEYDOWN:
    game.handle_keydown(event.key)
```

So the controls become:

‹/› Plain text

```
R → New Round
     Board clears
     Score stays

M → Reset Match
     Board clears
     X wins = 0
     O wins = 0
     Draws = 0
```

For example:

‹/› Plain text

```
X Wins: 2    O Wins: 1    Draws: 3
```

After pressing **R**, it remains:

‹/› Plain text

```
X Wins: 2    O Wins: 1    Draws: 3
```

After pressing **M**, it becomes:

‹/› Plain text

```
X Wins: 0    O Wins: 0    Draws: 0
```

This also preserves the three fixes from the previous version: **diagonals work, a last-move win beats a draw, and the board locks immediately when a round ends.**

> The scoreboard works but the text is too long for the window so Draws: gets cut off on the right side. Can you make it fit full.. Shorter labels or a smaller font are both fine. Full updated file please.

Absolutely — easiest clean fix is to use shorter labels: `X: 0 O: 0 D: 0`. That keeps the scoreboard fully visible without changing the rest of the UI.

Replace your complete `game_engine.py` with this:

`tic-tac-toe/game/game_engine.py`

```python
"""
GameEngine: owns the board, turn state, round-end logic,
and persistent match scoreboard.

Player = X
Computer = O
R = New round
M = Reset entire match
"""

import pygame

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move


HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'


class GameEngine:
    def __init__(self):
        # Current round
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = HUMAN_SYMBOL
        self.round_over = False
        self.winner = None
```

The scoreboard will now appear as:

`X: 2 O: 1 D: 3`

instead of the longer `X Wins: 2 O Wins: 1 Draws: 3`, so it should fit your existing window comfortably.

> last feature. i want to be able to choose whether X or O goes first. the computer always plays O so if O starts it should take the first move automatically. this should apply from the next round.
> i also want 2 separate controls one to restart just the current round keeping the scoreboard and one to reset the whole match clearing it. can u add clickable buttons for those plus a toggle for who goes first. the window might need to be taller to fit them.
> can u add this and just give comeplete updated files back

Yes — I'd make the controls part of the actual Pygame UI and keep the keyboard shortcuts too.

The updated version below gives you:

- **First: X / First: O** clickable toggle
- Toggle changes **who starts the next round**, not the current round
- If **O is selected**, the computer automatically makes the first move when the next round starts
- **New Round** button → clears only the board, keeps scoreboard
- **Reset Match** button → clears board + scoreboard

- `R` still works for New Round
- `M` still works for Reset Match
- Larger window so everything fits
- Existing diagonal/win/draw/input-lock fixes preserved

I checked the repo's current `main.py` and `renderer.py` structure so these files match the lab rather than changing the project architecture.

GitHub +1

`tic-tac-toe/main.py`

```python
"""
Tic-Tac-Toe (Lab)

Run with:
    python3 main.py

Controls:
    Click board cells to play X.
    R = New Round
    M = Reset Match

The UI also provides clickable controls for:
    - Choosing who starts the next round
    - Starting a new round
    - Resetting the entire match
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()

    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Tic-Tac-Toe")

    clock = pygame.time.Clock()
```

`tic-tac-toe/game/game_engine.py`

```python
"""
GameEngine: owns the board, turn state, round-end logic,
persistent match scoreboard, and UI controls.

Player = X
Computer = O

R = New Round
M = Reset Match
"""

import pygame

from game.rules import check_winner, is_board_full
from game.renderer import (
    board_pos_to_cell,
    START_TOGGLE_RECT,
    NEW_ROUND_RECT,
    RESET_MATCH_RECT,
)
from game.ai import choose_move


HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'
```

```
class GameEngine:
    def __init__(self):
```

tic-tac-toe/game/renderer.py

Python

```
"""
renderer: all pygame drawing lives here,
kept separate from game logic.
"""

import pygame



# ----------------------------------
# WINDOW / BOARD
# ----------------------------------

WIDTH, HEIGHT = 400, 630

BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100

WINDOW_SIZE = (WIDTH, HEIGHT)



# ----------------------------------
# COLORS
# ----------------------------------

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)

COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
```

tic-tac-toe/game/rules.py

No changes are required to the rules from the previous fix, but here is the complete file so you have the full updated set:

Python

```
"""
rules: win/draw detection for a 3x3 board.

board is a 3x3 list of lists, each cell is None, 'X', or 'O'.
"""



def check_winner(board):
    lines = []

    # Rows
    for r in range(3):
        lines.append([
            board[r][0],
            board[r][1],
            board[r][2]
        ])

    # Columns
    for c in range(3):
        lines.append([
            board[0][c],
            board[1][c],
            board[2][c]
        ])

    # Main diagonal
    lines.append([
        board[0][0],
        board[1][1],
```

The UI will now look roughly like:

```
┌──────────────────────────────────────┐
│ X: 2    O: 1    D: 3                 │
│                                      │
│        ┌───┬───┬───┐                 │
│        │ X │ O │   │                 │
│        ├───┼───┼───┤                 │
│        │   │ X │   │                 │
│        ├───┼───┼───┤                 │
│        │ O │   │ X │                 │
│        └───┴───┴───┘                 │
│                                      │
│ [ Next: X starts ] [ New Round ]     │
│ [ Reset Match ]                      │
│                                      │
│ Your turn (X)                        │
│ R: New Round    M: Reset Match       │
└──────────────────────────────────────┘
```

One important behavior: if you click `Next: X starts` and change it to `Next: O starts`, the current board does **not** suddenly change. When you press **New Round**, O immediately makes its first move. This keeps the "who goes first" choice specifically for the **next round** as requested.